Student Meal Planner

Project Reflection

A Personal Leadership Document for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

The Starting Point

When I started Grubs4Scrubs, I'd never built a full-stack web app. I knew some HTML and CSS. JavaScript was basic. C# was completely foreign. The idea of connecting a frontend to a backend to a database sounded straightforward in theory but I had no idea how much would go into actually making it work.

This reflection covers what I learned from the project specifically. Not the courses or the tutorials, but the lessons that came from building something real and hitting real problems.

Technical Growth

React: I went from not understanding what a component is to building multi-section pages with state management, routing, and interactive elements. The ingredient checklist on the Recipe View page, the day tab selector on the Meal Planner, the shopping list checkboxes on the Dashboard. These aren't complicated features, but building them from scratch taught me how React actually works. Props confused me until I had to pass data between components and saw what happens when you don't. useState confused me until I built the shopping list and watched checkboxes toggle in real time.

C# and ASP.NET Core: I went from zero C# to a layered architecture with repositories, services, and controllers. Writing the RecipeRepository with raw ADO.NET (SqlConnection, SqlCommand, SqlDataReader) taught me what an ORM actually abstracts away. Registering services with dependency injection taught me how the layers connect without knowing about each other. It's one of those concepts that sounds abstract until you see `builder.Services.AddScoped<IRecipeRepository, RecipeRepository>()` and realize the controller never creates a repository, it just asks for one. Building the AuthController taught me about security fundamentals: why you hash passwords instead of encrypting them, why BCrypt is better than SHA256 for passwords, how JWT tokens carry claims so the server doesn't need session state, and why you return the same error message for wrong email and wrong password (to prevent email enumeration).

CSS Architecture: The Tailwind removal taught me that picking a styling approach early matters more than which approach you pick. Custom CSS with `ComponentName-element-subelement` naming works for this project. Tailwind might work for others. The real lesson was about consistency: one system applied everywhere beats two systems applied randomly.

SQL and Database Design: Writing raw SQL forced me to think about what each query actually does. With EF Core, you write `_context.Recipes.ToList()` and SQL happens somewhere in the background. With ADO.NET, you write the SELECT statement, open the connection, execute the command, and map every column to a property. It's more work but I understand the database layer completely. No magic.

Non-Technical Growth

Decision making: The biggest non-technical skill I developed was making decisions and documenting why. Should I use Tailwind or custom CSS? Should the backend be one project or four? Should I use EF Core or ADO.NET (that one was decided for me, but I still had to understand why)? Each of these required research, comparison, and a choice. And each time, writing the reasoning down as a research doc made the decision clearer.

Problem solving under uncertainty: Half the errors I hit during development had unhelpful error messages. "Error Locating Server/Instance Specified" could mean twenty different things. A truncated JSX file just says "unterminated contents." Learning to systematically narrow down problems (check the connection string, check the imports, check the closing tags) was a skill I didn't have before this project.

Working alone: Solo projects are different from group projects. There's nobody to ask "does this look right?" before committing. Nobody catches your typos in namespace declarations. The upside is that every decision is yours and you understand the whole codebase. The downside is that every mistake is yours too. I got better at reviewing my own work and testing more carefully because there's no safety net.

[NOTE: Add any personal growth points that feel real to you. Confidence, communication with your coach, handling frustration, time management, etc.]

Mistakes That Taught Me the Most

The Tailwind mess: Starting two pages with different CSS approaches created a problem that took a full sprint to fix. Taught me to make foundational decisions before writing code.

The flat backend: Building everything in one project and then having to restructure into four projects mid-semester. Every file moved, every namespace changed, every reference updated. Taught me that architecture decisions made early are cheap, but made late are expensive.

The connection string: Hours lost to a wrong server format in a JSON config file. Taught me to verify infrastructure setup (database running, connection string correct, table exists) before writing any application code.

The stray import: One accidental import line in HomePage.jsx took down the whole app with a 500 error. Taught me to check imports carefully and test the full app after changes, not just the page I'm working on.

What I'm Proud Of

The codebase is clean. That matters to me. Every page follows the same CSS convention. The backend has clear layers with clear responsibilities. The research docs explain why decisions were made, not just what they are. If someone opened the Grubs4Scrubs repo tomorrow, they could understand how it's built and why it's built that way. That wasn't true a month ago.

[NOTE: Add anything else you're personally proud of.]

What's Next

The backend auth system is done (BCrypt hashing, JWT tokens, register and login endpoints). What's left is connecting the frontend login and signup pages to those endpoints, storing the token client-side, and sending it with API requests. The meal planner still needs to persist data to the backend instead of living in React state. Recipe ownership enforcement (only the creator can edit or delete) needs the auth token to be sent with requests first. Google OAuth is a could-have, not a must-have.

But the foundation is solid. The architecture is in place, the patterns are established, and adding new features now means following the patterns instead of inventing new ones each time. Adding the shopping list feature proved that: I built the entire backend (model, interface, repository, service, controller) and the full frontend (with inline editing) by following the same patterns as recipes. That's probably the most important thing I built this semester: not any single feature, but a codebase that's ready to grow.